

ADAPT2HIDE: LEVERAGING OFF-THE-SHELF AUTOENCODER FOR REVERSIBLE VISUAL PROCESSING

Ernie Chu^{*,1,2} I-Sheng Fang^{*,1} Tai-Ming Huang¹ Pin-Yen Chiu¹ Vishal M. Patel² Jun-Cheng Chen¹

¹Research Center for Information Technology Innovation, Academia Sinica ²Johns Hopkins University



Fig. 1: Our results on images in-the-wild. Considering various irreversible image processing operations, such as censorship and AI editing, our method, Adapt to Hide (Adapt2Hide), can hide the original image inside its processed version for reversible visual processing. The figure shows our reconstructions and the processed images. Full side-by-side comparisons can be found in Figure S.12.

ABSTRACT

Reversible visual processing, a novel task of image steganography by hiding and later receiving the original image from its processed version under various irreversible image processing operations, has diverse applications. Despite its potential, existing approaches are often specialized for specific tasks and face limitations in handling real-world degradations like image quantization, thus hindering their practical adoption. In this work, we find that large pretrained autoencoders (AEs) are surprisingly suitable for reversible visual processing. We thus introduce Adapt to Hide (Adapt2Hide), a novel reversible visual processing method leveraging off-the-shelf pretrained AEs and LoRA adapters. Unlike prior approaches, Adapt2Hide operates in the latent space of AEs, offering high-quality image reconstruction with minimal additional parameters (around 10% of the base AE) and efficient training with LoRA. Adapt2Hide’s design enables robust performance across a wide range of tasks, including reversible visual censorship and reversible AI editing, allowing users to retrieve the original image from censored or AI-edited images with competitive perceptual quality compared to specialized models. Additionally, Adapt2Hide demonstrates the robustness of common distortions such as cropping, quantization, and compression. Code

is at <https://github.com/aiiu-lab/Adapt2Hide>.

Index Terms— Reversible visual processing, Pretrained autoencoders, Latent-space modeling, LoRA, Image editing.

1. INTRODUCTION

The manipulation and generation of visual content have become crucial in various domains. However, reversible visual processing does not receive enough attention. As shown in Figure 1, reversible visual processing involves hiding an original image within a processed image, avoiding the need for a separate transmission to retrieve the original image, especially useful when transmission capabilities are limited. For example, Alice and Bob work in a remote location with limited bandwidth. Sending multiple large files is inefficient. Alice can hide an original image in the edited image and send it to Bob. If Bob later needs the original image for further editing, he can use reversible visual processing to extract the hidden (original) image from the container (edited) image. Specifically, the processing could be style transfer [2], halftone [3], grayscale [4], mononization [5], (visual) censorship, or AI editing. Essentially, reversible visual processing can be viewed as a type of steganography [6–16].

Current methods [2–5, 10, 12, 17] for image steganography are

^{*}Equal contribution.

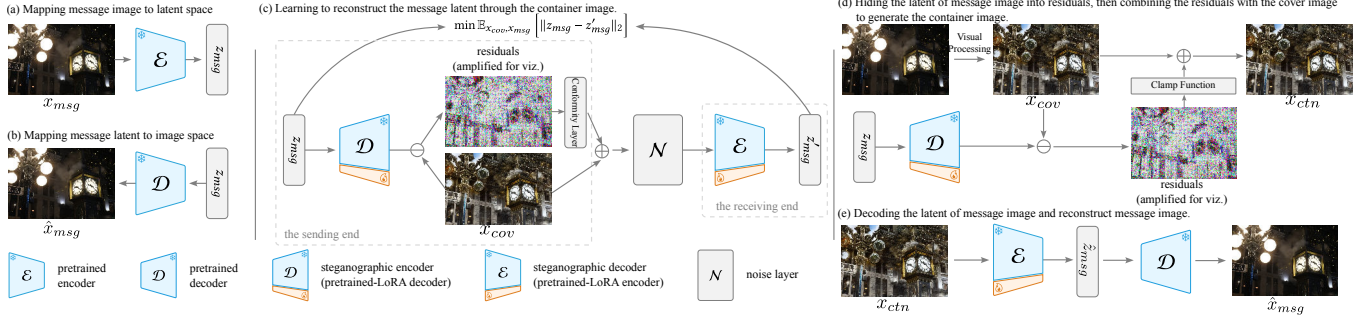


Fig. 2: Adapt2Hide can hide a message image x_{msg} inside a container image x_{ctn} , making it hard to distinguish from a cover image x_{cov} . This figure demonstrates reversible AI editing as an example application, where we hide the original image inside an AI-edited image to ensure editing traceability. (a,b) Our steganographic networks operate in the latent space defined by an AE $\{\mathcal{E}, \mathcal{D}\}$. (c,d) With tunable LoRA adapters [1], $\mathcal{D}$ functions as our steganographic encoder, while $\mathcal{E}$ acts as the decoder. The information of x_{msg} is embedded through imperceptible residuals (Sec. 2.2), which we amplified 100 times for visualization. Optionally, several noise layers $\mathcal{N}$ can be applied during training to further strengthen the container image against various distortions (Sec. 2.3). The entire pipeline is trained with a simple L2 loss. (e) To retrieve x_{msg} , Adapt2Hide decodes the latent representation $\hat{z}_{msg}$ from x_{ctn} and reconstructs x_{msg} .

task-specific and have limited adaptation. The limited adaptation of current methods can be attributed to several factors. Some methods [2, 10, 12] do not explicitly discuss basic degradations such as 8-bit unsigned integer quantization, which is crucial for distributing container images¹ in common image formats, limiting their practical utility. Additionally, these methods are usually built from scratch and specialized for a single application, such as style transfer [2], halftone [3], grayscale [4], and mononization [5], making them difficult to adapt to other applications. Moreover, the training loss design of these methods usually has conflicting terms, which need to be weighted, increasing the training complexity.

To address these limitations, we propose the method, Adapt to Hide (Adapt2Hide), by leveraging the latent space and model weights of large off-the-shelf pretrained autoencoders (AEs) [18–20] for reversible visual processing. These AEs are widely used as first-stage compressors in diffusion/autoregressive generators [20–25] and are often trained with perceptual/adversarial objectives for detail preservation [26–28]. Unlike previous methods, Adapt2Hide reconstructs messages within the latent space of an autoencoder (AE). Our approach allows for high-quality retention of fine details with only requiring approximately 10% more parameters than base AE and a simple L2 loss function for efficient training with LoRA [1]. Thanks to these powerful off-the-shelf AEs and LoRA adapters [1], our method works robustly on various applications. Additionally, our special design, which uses a clamp function to produce the container image, avoids conflicting loss terms and the need to weight them. Moreover, unlike existing methods that require both the cover and message image as input to their steganographic encoder, our encoder only requires the message images as input. This design is essential for using the off-the-shelf pretrained AE in image steganography, as the AE also takes one image as input.

In this paper, we introduce two rarely discussed important subtasks of reversible visual processing: reversible visual censorship and reversible AI editing. In reversible visual censorship, our method allows for the faithful recovery of original content from censored images, exposing potential vulnerabilities in third-party censorship services. In reversible AI editing, Adapt2Hide embeds the original image within its AI-edited version, optimizing data

usage with limited bandwidth. Reversible AI editing also could be used to provide traceability and reduce potential misuse. Additionally, our method includes modules to enhance robustness against common image distortions such as cropping, quantization, and compression. The results show that Adapt2Hide offers a reliable solution for reversible visual processing.

Our contribution can be summarized as follows:

- We leverage off-the-shelf pretrained AEs and LoRA adapters for reversible visual processing, enabling high-quality message reconstruction with minimal additional parameters.
- We introduce a novel design that simplifies training by using a clamp function to produce the container image, avoiding the need for conflicting loss terms and input dependencies seen in previous methods.
- Our method is easy to adapt to different applications by switching the LoRA adapter, extending reversible visual processing to novel areas like reversible visual censorship and reversible AI editing, while providing robustness against common distortions.

2. METHOD

Given a cover image x_{cov} and a message image x_{msg} , our method can hide x_{msg} inside a container image x_{ctn} that is visually indistinguishable from x_{cov} . After x_{ctn} is distributed to the public domain, anyone with access to our steganographic decoder can confidently recover x_{msg} from x_{ctn} . For a more concrete example, Figure 2 showcases an example on reversible² AI editing and briefly previews our system design.

Our method leverages the latent space of a well-pretrained autoencoder (AE). Let $\mathcal{E}$ and $\mathcal{D}$ represent the encoder and decoder of the AE, respectively. Given an in-distribution sample x , the AE satisfies the following equations:

$$z = \mathcal{E}(x), \quad (1)$$

$$x = \mathcal{D}(z) + \xi, \quad (2)$$

where z is the latent-space representation of x , and ξ is a small estimation error depending on the model’s performance. To transform these networks into our steganographic encoder and decoder,

¹A good container image looks similar to the cover image but with the message image hidden in it.

²By “reversible,” we mean that perfect reconstruction is not mandatory. Instead, we aim for perceptually identical image reconstruction.

respectively, we employ trainable LoRA adapters on both networks, denoted as $\tilde{\mathcal{E}}$ and $\tilde{\mathcal{D}}$.

Notably, we use $\tilde{\mathcal{E}}$ as the generally-perceived steganographic DECODER and $\tilde{\mathcal{D}}$ as the steganographic ENCODER. Under this seemingly absurd setting, we can leverage the powerful high-frequency modeling capability of the large pretrained AE. Specifically, x_{msg} is first encoded into a latent message $z_{msg} = \mathcal{E}(x_{msg})$ using the original pretrained encoder. Then, we produce the container image using the LoRA-patched decoder, *i.e.*

$$x_{ctn} = (\mathcal{N} \circ \mathcal{C})(x_{cov}, \tilde{\mathcal{D}}(z_{msg})), \quad (3)$$

where $\mathcal{C}$ is a conformity layer (Section 2.2) that ensures x_{ctn} conforms to the cover image x_{cov} , and $\mathcal{N}$ represents one or more noise layers (Section 2.3).

With access to $\mathcal{D}$, the ability to reconstruct z_{msg} allows for the recovery of x_{msg} , since $x_{msg} = \mathcal{D}(z_{msg}) + \xi$. Therefore, aiming to decode z_{msg} from x_{ctn} , a simple MSE photometric loss can effectively supervise the training of the LoRA:

$$\min_{\tilde{\mathcal{E}}, \tilde{\mathcal{D}}} \mathbb{E}_{x_{cov}, x_{msg}} [\|z_{msg} - z'_{msg}\|_2], \quad (4)$$

where $z'_{msg} = \tilde{\mathcal{E}}(x_{ctn})$. While prior works in the field typically use an adversarial loss to model high-frequency details [11] and have empirically shown that pixel error losses alone result in blurry images [28], our research demonstrates that naive regression is sufficient if you have a powerful decoder capable of producing sharp images from low-dimensional latents. To sum up, the final recovered image $\hat{x}_{msg}$ is obtained through a symmetric inference pipeline,

$$\hat{x}_{msg} = \mathcal{D}(\tilde{\mathcal{E}}(\mathcal{C}(x_{cov}, \tilde{\mathcal{D}}(\mathcal{E}(x_{msg}))))), \text{ c.f. Figure 2.} \quad (5)$$

2.1. Straight-through estimator

In the following sections, we incorporate the straight-through estimator [29],

$$\text{STE}(x, f) = x + \text{StopGrad}(f(x)) - \text{StopGrad}(x), \quad (6)$$

to handle non-differentiable operations, such as clamping and rounding.³ In short, the first and final terms cancel out in the forward pass, so $f(x)$ is passed down. In the backward pass, only the first term connects to the previous part of the computation tree, and backpropagation continues from there, *i.e.* $d \text{STE}(x, f)/dx = 1$.

2.2. Conformity layer

Our method is effective only when the container image x_{ctn} is visually indistinguishable from the cover image x_{cov} . To achieve this, we introduce the conformity layer. Unlike traditional methods that incorporate this objective as a loss term, we use a clamp function to keep the steganographic signal within a predefined range of $[-\epsilon, \epsilon]$, ensuring imperceptible perturbation. We define conformity layer as:

$$\mathcal{C}(x_{tgt}, x_{src}) = x_{tgt} + \text{STE}(x_{src} - x_{tgt}, \text{clamp}), \quad (7)$$

where the clamp function is:

$$\text{clamp}(x) = \begin{cases} \epsilon, & \text{if } x \geq \epsilon \\ -\epsilon, & \text{if } x \leq -\epsilon \\ x, & \text{otherwise} \end{cases} \quad (8)$$

We usually set the budget $\epsilon = 2$ under the pixel range of $[0, 255]$. The intuition behind this is that since the last term of Equation 7 is small, the output of the conformity layer, dominated by x_{tgt} , would be an in-distribution sample with respect to $\tilde{\mathcal{E}}$. Additionally, LoRA adapters have no effect at the beginning of training due to the zero initialization. Consequently, $\tilde{\mathcal{E}}(x_{ctn})$ would initially be z_{cov} and

would gradually transition to z_{msg} , with a stable update trajectory for $\tilde{\mathcal{E}}$ and $\tilde{\mathcal{D}}$ throughout the training. This process results in all outputs being reasonably high-quality images.

2.3. Noise layer

In real-world practice, our model should robustly reconstruct x_{msg} against various image distortions on x_{ctn} . In this section, we introduce different types of noise layers, denoted as $\mathcal{N}_{\text{type}}$. We implement $\mathcal{N}$ in Equation 3 by applying these noise layers sequentially.

Quantization. Portable images are often stored in a limited number of bytes, whereas neural networks are usually trained with much higher precision, requiring more bytes. Therefore, it is essential to consider certain types of image quantization during training. Specifically, we must address quantization in the spatial domain (e.g., PNG) and the frequency domain (e.g., JPEG).

Spatial domain. Abiding by common PNG specifications, each pixel p per RGB channel is stored as an 8-bit unsigned integer, *i.e.* $p \in [0, 255] \cap \mathbb{Z}$. The noise layer for spatial domain quantization is defined as follows:

$$\mathcal{N}_{sq}(x) = \text{STE}(x, \text{round}), \quad (9)$$

where $\text{round}(x) = \lfloor x \rfloor$.

Frequency domain. We define the noise layer for frequency domain quantization as follows⁵:

$$\mathcal{N}_{fq}(x, m) = \text{iDCT}(\text{STE}(\text{DCT}(x)/m, \text{round})), \quad (10)$$

where m is a 8×8 quantization matrix typically derived from a user-specified quality factor q . Here, DCT and iDCT correspond to the Discrete Cosine Transform and inverse Discrete Cosine Transform, respectively. Notably, m is unknown to the steganographic encoder $\tilde{\mathcal{D}}$, and different values of m create different information bottlenecks. How to enable $\tilde{\mathcal{D}}$ to find the intersection of all information bottlenecks (if it exists) is an ongoing problem and beyond the scope of this paper. Here, we simply consider two practical cases: lossy compression with $q = 90$ and lossless compression. In addition to $\mathcal{N}_{fq}$, we also consider a real JPEG compressor to mitigate the train-test discrepancy in DCT operations, referred to as $\mathcal{N}_{fqr}$. In this case, STE is applied to the entire JPEG encoding-decoding process rather than just the quantization:

$$\mathcal{N}_{fqr}(x, m) = (\text{STE}(x, \text{JPEG})). \quad (11)$$

Cropping With any fully-convolutional pretrained autoencoder, cropping operations on x_{ctn} naturally affect the message reconstruction $\mathcal{D} \circ \tilde{\mathcal{E}}(x_{ctn})$. However, due to the latent conversion in our design, we empirically find that the upper-left coordinate of a crop box must be multiples of the latent downscale factors f_H and f_W . Otherwise, the reconstruction corrupts. Given an image with dimensions of (C_i, H, W) , $\mathcal{E}$ produces a latent representation with dimension of $(C_l, \lfloor H/f_H \rfloor, \lfloor W/f_W \rfloor)$. We hypothesize that a grid-like pattern, imperceptible to the human eye, is imprinted on the output images, and the trainable networks $\tilde{\mathcal{E}}$ and $\tilde{\mathcal{D}}$ are sensitive to this pattern. To address this issue, we propose $\mathcal{N}_{crop}$, which randomly crops images using a crop box with the upper-left coordinate of (x_1, y_1) and the lower-right coordinate of $(x_1 + W - f_W, y_1 + H - f_H)$. Here, $x_1 \in [0, f_W]$, $y_1 \in [0, f_H]$ and $x_1, y_1 \in \mathbb{Z}$.

3. APPLICATION

In this section, we introduce two novel applications of reversible visual processing with our method. The first application addresses the

³We opt to use STE for simplicity, as other dedicated differentiable approximations for these operations are not our focus.

⁴We ignore the effect of $\mathcal{N}$ here for brevity.

⁵We omit color space conversion and chroma subsampling for brevity.

potential vulnerabilities caused by unwarranted trust in image censorship. The second application offers traceability for GenAI image editing by embedding source images within the edited versions. We will use a general equation,

$$x_{cov} = \mathcal{F}_{app}(x_{msg}, c_{app}), \quad (12)$$

to describe the relationship between x_{cov} and x_{msg} in each task. Here, $\mathcal{F}_{app}$ performs different operations depending on the task and may accept one or multiple configuration options c_{app} . Considering the potential for both beneficial and harmful uses, we believe it is crucial for the community to be fully aware of this technology.

3.1. Reversible visual censorship

Visual censorship is widely used in various industries. With the proposed method, we demonstrate that imperceptible perturbations on censored images can be exploited to recover the original content. This raises concerns about the trustworthiness of third-party censorship services, even if they are offline and their censored images appear visually satisfactory. We consider three common modes of image censorship: pixelation, blur, and black. Specifically, given a source image x_{src} and a mask $\mathcal{M}$, we obfuscate the area specified by $\mathcal{M}$ to obtain the censored image x_{csr} with

$$x_{csr} = \mathcal{F}_{censor}(x_{src}, \mathcal{M}, m, w), \quad (13)$$

$$= \mathcal{M} \otimes f(x_{src}, m, w) + (1 - \mathcal{M}) \otimes x_{src}, \quad (14)$$

where

$$f(x, m, w) = \begin{cases} \text{AvgPool}(x, w), & \text{if } m \text{ is pixelation} \\ \text{GaussianBlur}(x, w), & \text{if } m \text{ is blur} \\ 0x, & \text{if } m \text{ is black} \end{cases}, \quad (15)$$

w refers to block width or kernel width, depending on the context, and $\otimes$ denotes the Hadamard product. Instead of directly providing users with x_{csr} , a malicious image censorship service may hide x_{src} in a visually identical alias x'_{csr} such that

$$x'_{csr} = \mathcal{C}(x_{csr}, \tilde{\mathcal{D}}(\mathcal{E}(x_{src}))). \quad (16)$$

The sensitive information in x_{src} is exposed to those who have access to $\tilde{\mathcal{E}}$ without alerting the users.

3.2. Reversible AI editing

Recent advancements in generative visual models have shown remarkable capabilities in photorealistic image editing, featuring intuitive interfaces that accept simple natural language instructions. Inspired by food traceability systems that reassure consumers about the origin and history of food products, we propose a similar approach for images. We embed the original images into edited ones to mitigate potential misuse. In a simplified formulation, the edited images x_{edited} can be expressed as

$$x_{edited} = \mathcal{F}_{edit}(x_{src}, p), \quad (17)$$

where p is a natural language prompt specifying the desired visual manipulation on x_{src} . Before giving x_{edited} directly to users, we recommend that these tools use our method to always embed x_{src} on the edited images:

$$x'_{edited} = \mathcal{C}(x_{edited}, \tilde{\mathcal{D}}(\mathcal{E}(x_{src}))). \quad (18)$$

This ensures that AI-generated images can trace back to their authentic source images in case of potential misuse.

4. EXPERIMENTS

All evaluations focus on two criteria: **1) Imperceptibility:** How well does an encode cover image resemble to the clean cover image. **2) Reversibility:** How well does an reconstructed message image resemble to the original message image.

4.1. Implementation detail

For the pretrained AE, we use the VAE from Stable Diffusion v1.5.⁶ We also use the mask generator from LAMA [32] for reversible visual censorship. During Training, we use the degraded version of z_{msg} , denoted as $\hat{z}_{msg} = \mathcal{E}(\mathcal{N}(x_{msg}))$, which has the same distortion in x_{ctn} introduced by $\mathcal{N}$. For JPEG operations, we use TorchJPEG’s implementation [33] of DCT operations for $\mathcal{N}_{fq}$, and Pillow’s JPEG⁷ for $\mathcal{N}_{q,s}$ and testing. The budget ϵ is set to 2, and all models are optimized with 5,000 update steps and a batch size of 128 using Prodigy [34] on 512×512 images unless stated otherwise. LoRA rank is set to 64 in all layers, resulting in 13% additional trainable parameters. Training a model takes around 12 hours on 16 NVIDIA V100 32GB GPUs. For an 1024×1024 image, it takes 0.781 ± 0.209 seconds for encoding (i.e., $\mathcal{E}(\cdot)$ and $\tilde{\mathcal{D}}(\cdot)$) and 0.713 ± 0.181 seconds for decoding (i.e., $\mathcal{D}(\cdot)$ and $\tilde{\mathcal{E}}(\cdot)$) during inference in a NVIDIA A5000 24GB GPU during inference under 20 trials.

4.2. Metric

Our evaluation includes quantitative, qualitative, and subjective results. For the quantitative evaluation, it’s important to note that the off-the-shelf pretrained VAE, which uses adversarial loss during training, generates high-quality images but achieves a low PSNR when its reconstructed images are compared with the the input images. In other words, the reconstruction performance of recovered message images $\hat{x}_{msg}$ is limited by the off-the-shelf VAE. This limitation makes direct comparison between our recovered message image $\hat{x}_{msg}$ and the original message image x_{msg} less meaningful because Adapt2Hide hides and recovers the latent z_{msg} of the original message image x_{msg} . Therefore, for a fair evaluation of Adapt2Hide’s reversibility, we use the VAE-reconstructed images $\mathcal{D}(\mathcal{E}(x))$ as the ground truth of the recovered message images $\hat{x}_{msg}$ in our main experiment to obtain more meaningful results.

In addition to common metrics like PSNR, SSIM, and LPIPS, we also introduce novel perceptual metrics, SD-Sim and SDXL-Sim, to evaluate the performance of latent reconstruction and image reconstruction at the perceptual level. Both metrics are the cosine similarity of two normalized vectors⁸ in the latent space of the VAE of Stable Diffusion v1.5 [20] or Stable Diffusion XL [35] respectively. Finally, to better reflect visual perception, we conduct a user study and denote the results as “Human rating”. In our user study, participants rated the similarity between message and recover images on a scale of 1 to 5 (from most dissimilar to most similar). Each participant completed up to 30 rounds, with the option to stop at any time. We collected 3,660 votes from 122 participants.

4.3. Main result

For reversible visual censorship, we train our model on ImageNet-1k [30] and evaluate it on the validation set. The censored area is specified by a random mask, and the censor mode is randomly selected from pixelation, blur, or black. For reversible AI editing, one can perform editing on the fly using off-the-shelf models such as InstructPix2Pix [31]. However, to save training time, we use the same dataset⁹ on which InstructPix2Pix was trained. We randomly

⁶<https://huggingface.co/runwayml/stable-diffusion-v1-5>

⁷<https://python-pillow.org>

⁸Image latents are flattened to 1D vectors

⁹<https://huggingface.co/datasets/timbrooks/instructpix2pix-clip-filtered>

Task	Method	Reversible visual censorship (ImageNet-1K [30])					Reversible AI editing (InstructPix2Pix [31])				
		PSNR [↑]	SSIM [↑]	LPIPS [↓]	SD-Sim [↑]	SDXL-Sim [↑]	PSNR [↑]	SSIM [↑]	LPIPS [↓]	SD-Sim [↑]	SDXL-Sim [↑]
Clean	Ours w/o $\mathcal{N}_{sq}$	31.97	0.9378	0.0480	0.9908	0.9350	29.15	0.9337	0.0496	0.9878	0.9322
	Ours	32.90	0.9545	0.0417	0.9925	0.9365	32.46	0.9577	0.0341	0.9951	0.9545
	$\mathcal{D}(\mathcal{E}(\cdot))^*$	28.07	0.7847	0.1324	0.9899	0.8352	28.40	0.8172	0.1051	0.9916	0.8939
JPEG	Ours	26.17	0.8027	0.0373	0.9933	0.9404	18.87	0.6129	0.8111	0.0211	0.0423
	Ours w/ $\mathcal{N}_{fq}$	29.25	0.8895	0.0596	0.9881	0.9215	19.15	0.6420	0.7217	0.4205	0.3720
	Ours w/ $\mathcal{N}_{fqr}$	29.99	0.8957	0.0601	0.9891	0.9229	21.65	0.7011	0.3933	0.8482	0.7190
	$\mathcal{D}(\mathcal{E}(\cdot))^*$	28.13	0.7878	0.1326	0.9903	0.8360	28.26	0.8134	0.1190	0.9915	0.8772
Cropping	Ours	15.56	0.4307	0.5036	0.5804	0.4713	7.75	0.1552	0.7607	0.1220	0.0395
	Ours w/ $\mathcal{N}_{crop}$	26.56	0.8153	0.1729	0.8977	0.7844	22.52	0.7177	0.2371	0.8409	0.6538
	$\mathcal{D}(\mathcal{E}(\cdot))^*$	27.83	0.7734	0.1344	0.8638	0.7160	27.47	0.7780	0.1266	0.8191	0.6294

* Comparing VAE-reconstructed images to the original reference images.

Table 1: Reversibility on censorship and editing. Imperceptibility is constant to the budget ϵ . Please refer to Table S.2 in appendix.

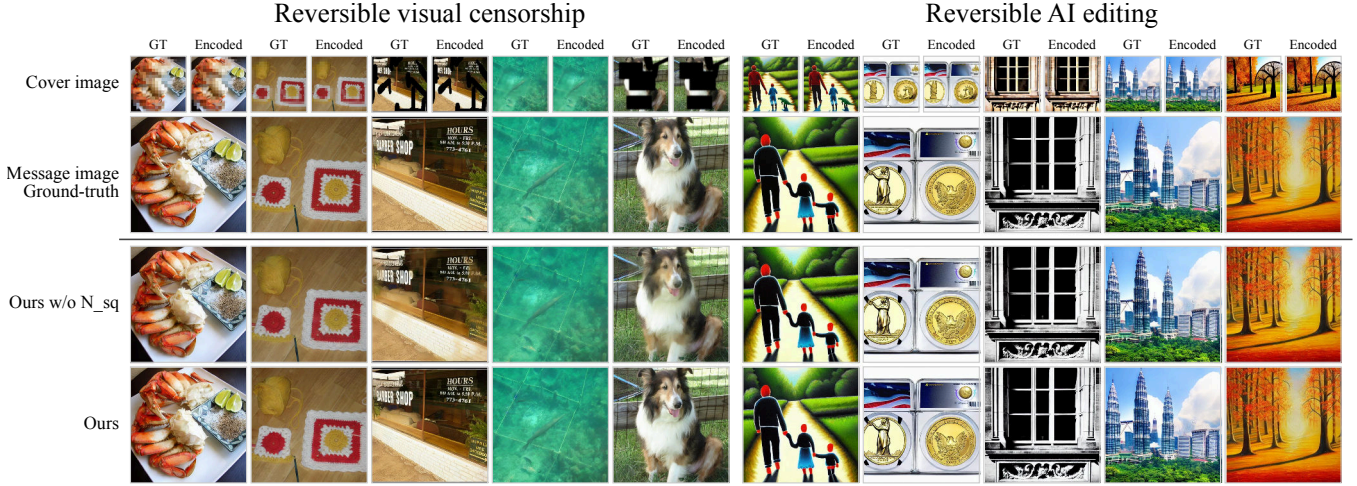


Fig. 3: Uncurated images under clean setting sampled with random seed 123. From left to right: reversible visual censorship on ImageNet-1k [30], reversible AI editing on InstructPix2Pix [31].

split the dataset into 9:1 training and testing sets. We consider three scenarios in our evaluation: Clean, JPEG, and Cropping. In the Clean setting, the encoded cover images are saved in PNG format and loaded back for message decoding. In the JPEG setting, half of the encoded cover images are further saved in JPEG with $q = 90$. In the Cropping setting, we randomly crop the encoded cover images. Since the encoded cover images are saved in 8-bit unsigned integers in all settings, $\mathcal{N}_{sq}$ is incorporated in all model variants. Other noise layers are used depending on the settings.

As we can clearly see from Table 1, and Figure 3, the model with $\mathcal{N}_{sq}$ provides slightly better results in the Clean setting. This difference is not very prominent because most of the steganographic perturbations are clamped to ± 2 , which are already integers. In the JPEG setting, the model with $\mathcal{N}_{fqs}$ performs better than the one with $\mathcal{N}_{fq}$. This shows that using STE through real JPEG compression during training is simple but effective. The model without either noise layer presents corrupted results. In the Cropping setting, the model without $\mathcal{N}_{crop}$ is severely corrupted, as discussed in Section 2.3. However, $\mathcal{N}_{crop}$ greatly mitigates this issue. For more results figures and ablation studies, please refer to Appendix.

5. CONCLUSION

Adapt2Hide demonstrates its effectiveness in the field of reversible visual processing by leveraging the power of large off-the-shelf pretrained AEs and LoRA adapters, showing significant promise for applications like visual censorship and AI editing. By utilizing the latent space of AE, Adapt2Hide achieves high-quality image reconstruction with minimal additional parameters, maintaining high-frequency details despite using a simple L2 loss function. Adapt2Hide’s application in AI-edited content allows users to embed the original image within its edited version. This application not only enables users to retrieve the original image from the edited one, even when bandwidth is limited, but also ensures traceability of the content. The method’s robustness against image distortions such as cropping, quantization, and compression underscores its practical utility. Overall, with its efficient parameter usage, adaptability, and robustness, Adapt2Hide demonstrates its broad applications.

Compliance with Ethical Standards. This research was conducted using open datasets. The user study was approved by the Institutional Review Board for Humanities & Social Science Research, Academia Sinica, under protocol number AS-IRB-HS-24031.

Acknowledgment

This research is supported by the National Science and Technology Council (NSTC), Taiwan under Grants 114-2221-E-001-004, 114-2221-E-001-016, 114-2634-F-001-001-MBK, 114-2634-F-002-004, 113-2634-F-002-008 and by Academia Sinica under Grants AS-IAIA-114-M10 and AS-GCS-115-M06. We thank to National Center for High-performance Computing (NCHC) of National Applied Research Laboratories (NARLabs) in Taiwan for providing computational and storage resources.

6. REFERENCES

- [1] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen, “LoRA: Low-rank adaptation of large language models,” in *ICLR*, 2022.
- [2] Hung-Yu Chen, I-Sheng Fang, Chia-Ming Cheng, and Wei-Chen Chiu, “Self-contained stylization via steganography for reverse and serial style transfer,” in *WACV*, March 2020.
- [3] Menghan Xia, Wenbo Hu, Xueting Liu, and Tien-Tsin Wong, “Deep halftoning with reversible binary pattern,” in *ICCV*, October 2021, pp. 14000–14009.
- [4] Menghan Xia, Xueting Liu, and Tien-Tsin Wong, “Invertible grayscale,” *ACM TOG*, vol. 37, no. 6, 2018.
- [5] Wenbo Hu, Menghan Xia, Chi-Wing Fu, and Tien-Tsin Wong, “Mononizing binocular videos,” *ACM TOG*, vol. 39, no. 6, 2020.
- [6] R.G. van Schyndel, A.Z. Tirkel, and C.F. Osborne, “A digital watermark,” in *Proceedings of International Conference on Image Processing*, 1994, vol. 2, pp. 86–90 vol.2.
- [7] Tomáš Pevný, Tomáš Filler, and Patrick Bas, “Using high-dimensional image models to perform highly undetectable steganography,” in *Information Hiding*, 2010, pp. 161–177.
- [8] Vojtěch Holub and Jessica Fridrich, “Designing steganographic distortion using directional filters,” in *2012 IEEE International Workshop on Information Forensics and Security (WIFS)*, 2012, pp. 234–239.
- [9] V. Holub, J. Fridrich, and T. Denemark, “Universal distortion function for steganography in an arbitrary domain,” *EURASIP Journal on Information Security*, 2014.
- [10] Shumeet Baluja, “Hiding images in plain sight: Deep steganography,” in *NeurIPS*, 2017, vol. 30.
- [11] Jiren Zhu, Russell Kaplan, Justin Johnson, and Li Fei-Fei, “Hidden: Hiding data with deep networks,” in *ECCV*, September 2018.
- [12] Junpeng Jing, Xin Deng, Mai Xu, Jianyi Wang, and Zhenyu Guan, “Hinet: Deep image hiding by invertible network,” in *ICCV*, October 2021, pp. 4733–4742.
- [13] Youmin Xu, Chong Mou, Yujie Hu, Jingfen Xie, and Jian Zhang, “Robust invertible image steganography,” in *CVPR*, 2022.
- [14] Jiwen Yu, Xuanyu Zhang, Youmin Xu, and Jian Zhang, “CRoSS: Diffusion model makes controllable, robust and secure image steganography,” in *NeurIPS*, 2023.
- [15] Jiaming Song, Chenlin Meng, and Stefano Ermon, “Denoising diffusion implicit models,” *arXiv preprint arXiv:2010.02502*, 2020.
- [16] Tu Bui, Shruti Agarwal, Ning Yu, and John Collomosse, “Rosteals: Robust steganography using autoencoder latent space,” in *CVPRW*, 2023.
- [17] Xiujian Liang, Bingshan Liu, Qichao Ying, Zhenxing Qian, Hsunfang Cho, and Xinpeng Zhang, “Stylestegan: Leak-free style transfer based on feature steganography,” in *2023 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*. IEEE, 2023, pp. 1443–1450.
- [18] Diederik P. Kingma and Max Welling, “Auto-encoding variational bayes,” in *ICLR*, 2014.
- [19] Aaron van den Oord, Oriol Vinyals, and koray kavukcuoglu, “Neural discrete representation learning,” in *NeurIPS*, 2017, vol. 30.
- [20] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer, “High-resolution image synthesis with latent diffusion models,” in *CVPR*, 2022.
- [21] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis, “Align your latents: High-resolution video synthesis with latent diffusion models,” in *CVPR*, June 2023, pp. 22563–22575.
- [22] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever, “Zero-shot text-to-image generation,” in *ICML*, 2021, vol. 139, pp. 8821–8831.
- [23] Patrick Esser, Robin Rombach, and Björn Ommer, “Taming transformers for high-resolution image synthesis,” in *CVPR*, June 2021, pp. 12873–12883.
- [24] Arash Vahdat, Karsten Kreis, and Jan Kautz, “Score-based generative modeling in latent space,” in *NeurIPS*, 2021, vol. 34, pp. 11287–11302.
- [25] Lvmin Zhang and Maneesh Agrawala, “Transparent image layer diffusion using latent transparency,” *arXiv preprint arXiv:2402.17113*, 2024.
- [26] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *CVPR*, June 2018.
- [27] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio, “Generative adversarial nets,” in *NeurIPS*, 2014, vol. 27.
- [28] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros, “Image-to-image translation with conditional adversarial networks,” in *CVPR*, July 2017.
- [29] Yoshua Bengio, Nicholas Léonard, and Aaron C. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv preprint arXiv:1308.3432*, 2013.
- [30] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [31] Tim Brooks, Aleksander Holynski, and Alexei A. Efros, “Instruct-pix2pix: Learning to follow image editing instructions,” in *CVPR*, 2023.
- [32] Roman Suvorov, Elizaveta Logacheva, Anton Mashikhin, Anastasia Remizova, Arsenii Ashukha, Aleksei Silvestrov, Naejin Kong, Harshith Goka, Kiwoong Park, and Victor Lempitsky, “Resolution-robust large mask inpainting with fourier convolutions,” in *WACV*, January 2022, pp. 2149–2159.
- [33] Max Ehrlich, Larry Davis, Ser-Nam Lim, and Abhinav Shrivastava, “Quantization guided jpeg artifact correction,” in *ECCV*, 2020.
- [34] Konstantin Mishchenko and Aaron Defazio, “Prodigy: An expeditiously adaptive parameter-free learner,” *arXiv preprint arXiv:2306.06101*, 2024.
- [35] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach, “SDXL: Improving latent diffusion models for high-resolution image synthesis,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [36] Yingqian Wang, Longguang Wang, Jungang Yang, Wei An, and Yulan Guo, “Flickr1024: A large-scale dataset for stereo image super-resolution,” in *ICCVW*, Oct 2019.
- [37] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL Visual Object Classes Challenge 2012 (VOC2012) Results,” <http://www.pascal-network.org/challenges/VOC/voc2012/workshop/index.html>, 2012.
- [38] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick, “Microsoft coco: Common objects in context,” in *ECCV*, 2014, pp. 740–755.